



Fast and accurate approximation algorithms for computing floating point square root

Zbigniew Kokosiński¹ · Paweł Gepner² · Leonid Moroz² ·
Volodymyr Samotyy¹ · Mariusz Węgrzyn¹ · Nataliia Gavkalova²

Received: 3 February 2024 / Accepted: 3 September 2024 / Published online: 2 October 2024
© The Author(s) 2024

Abstract

The square root is one of the most used functions in many different engineering and scientific applications. We propose new methods for calculating the square root function that are based on the Newton–Raphson method with Heron iteration. A modification of Heron’s formula combined with an improved selection of the magic constants enables a significant reduction of the maximum relative error (MRE). Simple modifications to the Newton–Raphson formula and the magic number method enable implementation on platforms with limited hardware resources, such as microcontrollers and FPGAs, with variable accuracy. Implementations of new approximation algorithms in the C programming language were carefully tested and evaluated against their software and hardware counterparts on the most popular platforms, e.g., CPUs from Intel, AMD and ARM, GPU from Nvidia and IPU from Graphcore. The proposed numerical algorithms are shown to be superior in terms of computational time, the number of clock cycles, accuracy, MRE, and root mean square deviation.

✉ Zbigniew Kokosiński
zk@pk.edu.pl

Paweł Gepner
pawel.gepner@pw.edu.pl

Leonid Moroz
leonid.moroz@pw.edu.pl

Volodymyr Samotyy
vsamotyy@pk.edu.pl

Mariusz Węgrzyn
mariusz.wegrzyn@pk.edu.pl

Nataliia Gavkalova
nataliia.gavkalova@pw.edu.pl

¹ Department of Automatic Control and Computer Engineering, Cracow University of Technology, Warszawska 24, Cracow 31-155, Poland

² Faculty of Mechanical and Industrial Engineering, Warsaw University of Technology, Narbutta 85, Warsaw 02-524, Poland

Keywords Numerical algorithm · Approximation algorithm · Function approximation · Square root · Newton–Raphson method · Heron formula

Mathematics Subject Classification (2010) 33F05 · 49M15 · 65Y10 · 68W25

1 Introduction

Computing the square root and other elementary functions are highly useful in various fields of science and engineering. Arithmetic operations are essential for high-performance computing (HPC), artificial intelligence (AI) including deep neural networks (DNN), digital signal processing (DSP), and in various scientific computing simulations and experiments, such as the Large Hadron Collider (LHC), where 25% of the simulation time was found to be spent in computing elementary functions (cf. [40]).

The basic methods for calculating the square root date back to ancient times. In this article we investigate the classical Newton–Raphson method [17] combined with the famous Heron formula for computing the square root function $y = \sqrt{x}$ for a floating-point argument x :

$$y_{n+1} = 0.5 \cdot (y_n + x/y_n) \quad (1)$$

This formula is an essential iterative step in a method invented by Heron of Alexandria, which starts with an initial approximation of y_0 (the seed), and repeats iteration (1) until the required accuracy is obtained. It is obvious that the computational effort of this method depends on the value of the seed, which should be as close to $\sqrt{x}$ as possible.

The main disadvantage of the Heron formula is the presence of the division operation, which has traditionally been considered complex, costly, and slow. However, given the advances in computer arithmetic (software and hardware components), the application of division operation in numerical computations should be re-examined in order to check whether we still need to avoid division in each iteration and rely exclusively on multiplications and additions.

We can think of multiplication as multiple additions and division as multiple subtractions. The difference between the two operations is that the additions in multiplication can be done in parallel, while division is inherently sequential: subsequent subtraction operations are preceded by comparisons. Thus, the hardware multiplier may exploit the inherent parallelism present in the multiplication operation by computing and summing up many subproducts simultaneously, at the cost of an increased die area, while the division operation does not have this feature.

If we take into consideration the first Pentium or Pentium MMX, then the floating point division operation was usually 8–13 times slower than addition or multiplication [18–21]. Moreover, hardware dividers were never pipelined, as that requires replicating a lot of circuitry instead of iterating on the same multiplier module [10]. Nevertheless, a lot of effort in the field of computer arithmetic has already been spent on simplifying or minimizing the number of operations that must be performed, including the

simplification of division. Furthermore, a wider usage of CPU pipelines with a higher instruction per cycle (IPC) factor shrinks the processor cycle gap and finally decreases the number of cycles needed for the completion of simple operations. Therefore, if the division operation is essential, it makes sense to add specialized hardware dividers. By now, specialized dividing circuits have become standard [44].

New generations of processors, such as the Intel Xeon Skylake, dramatically improve these scenarios. If we consider the worst case of addition or multiplication and the best case of division, we see the execution of single operations versus ‘pipelined throughput’. In this case, it’s 4–5 cycles for division operations versus one cycle for addition or multiplication [18]. So, we end up with a 4–5 times slower division versus addition or multiplication. Such a performance difference seems to be a significant improvement, and drastically increases the attractiveness of using division operations, especially when compared to the standard case [18, 21].

Taking into account this discussion, the Heron formula with division operations is still technically sound. It can be found in recent publications and it is a part of commercially used software. For instance, in the CMSIS DSP Software Library ([12] for STM32 microcontrollers) the way of computing the square root function is the classical Heron formula. The square root was also implemented in FPGA [4, 9, 10]. Therefore, further development of the theory of the Newton–Raphson method for computing the square root function is the subject of our research reported in the present article.

The contents of the rest of this article are as follows. Section 2 contains a survey of the research related to this paper’s topic. In Section 3, known approaches for computing the square root function are discussed. Two popular algorithms from the literature, referred to later in this paper, are recalled. In the next section, we present several new variants of algorithms as well as their characteristics in terms of accuracy (number of correct bits) and maximum and minimum relative errors. Four new algorithms with specific accuracy demands are given and used for comparison with their counterparts from the literature. Section 5 presents four other high-quality algorithms with suitable precision that do not involve the division operation. Section 6 presents an experimental verification of these new algorithms versus classical methods. The computations were performed on five platforms with CPU, GPU and IPU architectures from different vendors. The algorithms are compared in terms of execution time, the number of clock cycles, accuracy (the number of correct bits), root mean square deviation (RMSD), as well as the maximum and minimum relative errors. The conclusions from these research results are presented in the last section.

The code for all the algorithms proposed and referred to in the article as well as other useful data are freely available from our project in the GitHub repository.

2 Survey of the literature

Investigation of Heron formula has a long and distinguished history which can be traced in the vast literature on this topic: [2–6, 9–14, 16, 23, 25–27, 30–33, 37–39].

Modern methods for computing of square root include:

- direct look-up table and bipartite tables [1, 5, 30] - the method is good for low precision computations;
- digit recurrence methods [8] - these methods have linear convergence, but are rather slow;
- low-degree polynomial or rational approximations [1, 10, 22] - this method needs division and many add and multiply operations;
- multiplication-based methods: Newton–Raphson, Goldschmidt and Taylor algorithms [17, 26, 30, 43] - the algorithms include a seed generator (for instance LUT table), although they have quadratic convergence require many integer or floating-point (FP) arithmetic operations;
- methods that include multiplication-based magic constants (fast inverse square root) as initial guess and Newton–Raphson iterations and require many FP arithmetic operations [24, 28, 32–36, 39, 41];
- digit by digit, CORDIC method [5, 17, 30] - has linear convergence and is slow;
- a two-variable iterative method [15] - requires many arithmetic operations;

Several simulation approaches for choosing initial approximations of the seed (choosing an initial guess) were presented in [2, 27, 37]. An overview of this method can be found in [2, 11, 30, 37].

Surveys of floating-point arithmetic methods can be found in [7, 38].

3 Previous approximation algorithms

We know that the classical iterative method is Heron's rule, based upon the formula $f(y) = y^2 - x$, so that $f(\sqrt{x}) = 0$. The iteration applied in Newton–Raphson method is either

$$y_{n+1} = y_n - \frac{y_n}{f'(y_n)} = y_n - \frac{y_n^2 - x}{2 \cdot y_n}$$

or (1), which converges quadratically. Each iteration (1) costs one division operation, one addition, and one multiplication by 0.5.

Let us notice, that in practice, due to division operation, Heron formula is often replaced by a simpler multiplication-based Newton–Raphson formula (2) for computing the reciprocal square root:

$$y_{n+1} = 0.5 \cdot y_n \cdot (3 - x \cdot y_n \cdot y_n) \quad (2)$$

which contains only four multiplication operations and one subtraction, and enables easy way to obtain the square root function [1, 5, 21, 24, 26, 28, 29, 33–36, 38, 42].

Let us give a simple solution of iterative Heron formula (1) for initial approximation of y_0 in the closed form

$$y_n = \sqrt{x} \frac{(y_0 + \sqrt{x})^{2^n} + (y_0 - \sqrt{x})^{2^n}}{(y_0 + \sqrt{x})^{2^n} - (y_0 - \sqrt{x})^{2^n}}$$

Then, the current relative error equals

$$\delta_n = \frac{2 \cdot (y_0 - \sqrt{x})^{2^n}}{(y_0 + \sqrt{x})^{2^n} - (y_0 - \sqrt{x})^{2^n}}$$

Such a representation of formula (1), when the iteration number n is high, e.g. $n = 12$ [3, 4], enable time savings for computing of exact results and errors, which can reach sufficiently large values. Moreover, the formulas allow selection of the best initial approximation (constant, linear, quadratic, ...) in terms of the absolute error or the relative error for any iteration of Heron formula in the selected argument range. For example, if for initial approximation the value $y_0 = 0.5x$ is selected (cf. [12]) then the relative error of each iteration equals:

$$\delta_n = \frac{2 \cdot (0.5x - \sqrt{x})^{2^n}}{(0.5x + \sqrt{x})^{2^n} - (0.5x - \sqrt{x})^{2^n}} \quad (3)$$

Figure 1 shows the chart of the relative error $\delta_n(x)$ of the Heron method (1), according to formula (3), for $n = 1$ and $x \in [1, 16]$. As one can see, the function $\delta_n(x)$ is nonlinear within the argument range $[1, 16]$. The function is decreasing to 0 with x within the range $[1, 4]$. Then, it is increasing within the range $[4, 16]$. Therefore, in our opinion the best initial approximation of square root function for FP numbers is obtained by using so called magic numbers as it was proposed in [6, 16, 23, 39]. In particular, in [6] and [16] the Algorithm 1 was given for the initial approximation of FP square root. In comparison to the function depicted in Fig. 1 the function code of Algorithm 1 gives a still rough, but much more accurate initial square root approximation (MRE is about 0.06 for all normalized FP numbers).

In order to improve algorithm's accuracy, it is possible to specify a better magic constant 0x1fbb4ecc as shown in [39]. Additionally, we add to the code two Newton–

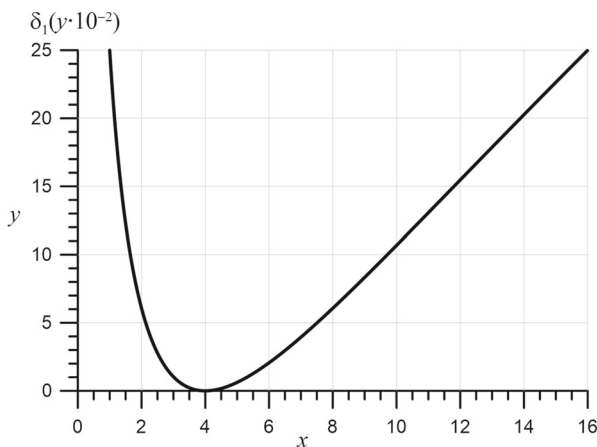


Fig. 1 The relative error of Heron method (1) given by (3): $\delta_1 = y(x) \cdot 10^{-2}$, for $n = 1$ and $x \in [1, 16]$

Raphson iterations (Heron formula), as proposed in [6]. Thus, we have Algorithm 2 referred to as Sqrt_Heron-Muller.

Algorithm 1 Sqrt_Blinn.

```
float Sqrt_Blinn(float x)
{ union { float x; unsigned i; } u ;
u.x = x ;
u.i = (u.i >> 1) + 0x1fc00000 ;
return u.x ; }
```

Algorithm 2 Sqrt_Heron-Muller.

```
float Sqrt_Heron-Muller(float x)
{ float y ;
int i = *(int*) & x ;
i = 0x1fbb4ecc + (i >> 1) ;
x = *(float*) & i ;
y = 0.5f * (y + x/y) ;
y = 0.5f * (y + x/y) ;
return y ; }
```

As we will show further, such magic number ensures obtaining the optimal MREs in the first and second iteration.

$$\begin{aligned}\delta_{0\min} &= -3.474e - 2; \delta_{0\max} = 3.474e - 2; \text{ (with 4.84 correct bits)} \\ \delta_{1\min} &= -5.069e - 8; \delta_{1\max} = 6.250e - 4; \text{ (with 10.64 correct bits)} \\ \delta_{2\min} &= -1.190e - 7; \delta_{2\max} = 2.559e - 7; \text{ (with 21.90 correct bits)}\end{aligned}$$

Throughout this paper pointers are used in order to simplify the algorithms' code, but in practice it is better to use the union data type.

4 New approach and algorithms

In our new approach to the application of Heron formula in conjunction with magic numbers we propose two significant improvements. To begin with, we propose a modification of the Heron formula

$$y_{n+1} = k_n \cdot (y_n + x/y_n), \quad (4)$$

where k_n - is a constant coefficient close to 0.5. This alteration provides an alignment of the maximal values of the positive and negative errors with respect to zero in each iteration. This trick may be used in conjunction with an improved selection of the magic constants in order to construct the initial approximation, which constitutes the second improvement. We will now explain the last issue in detail.

Recall, after [32], the basic theory and results related to the reciprocal square root approximation. It is known that the behaviour of the relative error in calculating y_0

over the whole range of normalized FP numbers can be described by its behaviour for $x \in [1, 4]$ [5, 30, 38]. In this range there are three piecewise linear approximations of the function $y_0 = \{y_{01}, y_{02}, y_{03}\}$:

$$y_{01} = \frac{t}{2N_m} + \frac{1}{4}x + \frac{1}{2} - \frac{1}{4N_m}, x \in [1, t), \quad (5)$$

$$y_{02} = \frac{t}{N_m} + \frac{1}{2}x - \frac{1}{2N_m}, x \in [t, 2), \quad (6)$$

$$y_{03} = \frac{t}{N_m} + \frac{1}{4}x + \frac{1}{2} - \frac{1}{2N_m}, x \in [2, 4), \quad (7)$$

where

$t = 2 - 2m_R + N_m^{-1}$; $m_R = N_m^{-1}R - \lfloor N_m^{-1}R \rfloor$, $m_R > 0$ – is the fractional part of the mantissa of the magic constant R ;

N_m – is the floating point precision of computations.

In order to find the optimal value of the magic constant from the point of view of minimizing the relative error of the initial approximation y_0 , consider the following maxima of relative errors $\delta_0 = \{\delta_{01}, \delta_{02}, \delta_{03}\}$:

$$\delta_{01} = y_{01}/\sqrt{x} - 1, \quad (8)$$

$$\delta_{02} = y_{02}/\sqrt{x} - 1, \quad (9)$$

$$\delta_{03} = y_{03}/\sqrt{x} - 1, \quad (10)$$

at the points $x_{\min} = t$, for y_{01} ; $x_{\max} = 2$, for y_{02} or y_{03} . Applying the method of [33], we find that the value $m_R = 0.463353872483829593509611$ is minimal for $N_m = 8388608$ (single precision). From this, we obtain $R_{0\min} = 0x1fbb4f2e$ (532369198), which only differs slightly from that given in [39], $0x1fbb4ecc$ (532369100), see the code for FP float in Sqrt_Heron-Muller, and gives almost the same MREs for the Heron formula in the first and the second iterations:

$$\delta_{1\min} = -5.069e - 8; \delta_{1\max} = -6.245e - 4; \text{ (with 10.64 correct bits)}$$

$$\delta_{2\min} = -1.1917e - 7; \delta_{2\max} = 2.534e - 7; \text{ (with 21.91 correct bits)}$$

We will continue to use this magic constant, $0x1fbb4f2e$, in the corresponding algorithm Sqrt_Blinn_Modified (available from our GitHub site). However, it is known from [2, 27, 30, 37] that for each iteration the optimal values of the relative error hold only for their initial approximations. This approach has been used to improve the method developed by [37]. Hence, the optimal initial approximation for the first iteration according to the classical Heron formula and the modified Heron code Sqrt_Modified_Heron_1 (available from our project repository on GitHub) has the same form as the Sqrt_Heron-Muller algorithm, only with a new magic constant. In this case, it is an improved magic constant $0x1fbb67af$. Below we will provide insight into its detailed computation.

Note the slightly lower values of the obtained errors in the first and second iterations

of the algorithm `SQRT_Modified_Heron_1`:

$$\begin{aligned}\delta_{1\min} &= -5.002e - 8; \delta_{1\max} = -6.009e - 4; \text{ (with 10.70 correct bits)} \\ \delta_{2\min} &= -1.1917e - 7; \delta_{2\max} = 2.530e - 7; \text{ (with 21.91 correct bits)}\end{aligned}$$

Modifying the Heron formula by changing the value of k_n makes it possible to significantly reduce the error of calculating the square root, especially when performing several Heron iterations. The influence of the value of k_n is stronger than the previously proposed approaches to reducing errors, such as an optimal choice of the initial approximation [2, 27, 30, 37]. For example, the new algorithm with one magic constant and a modified Heron formula (4) for the first iteration achieves much better properties of the relative error:

$$y_1 = k_0 * (y_0 + x/y_0). \quad (11)$$

Again, in this case we have to find an optimal value of the magic constant $R_{1\min}$, but from now on, together with the optimal value of the coefficient k_0 . For this purpose we shall list the general formulas for the relative errors in the three segments of the initial approximation y_0 , similarly to (8)–(10), but after iteration (11), substituting there (5)–(7), respectively:

$$\begin{aligned}\delta_{11} &= y_{11}/\sqrt{x} - 1; \\ \delta_{12} &= y_{12}/\sqrt{x} - 1; \\ \delta_{13} &= y_{13}/\sqrt{x} - 1,\end{aligned}$$

where

$$y_{1i} = k_0 * (y_{0i} + x/y_{0i}), i = 1, 2, 3.$$

Note that the coefficient k_0 in the Heron formula remains unchanged for all segments. Based on the maximum points, the values of the positive, negative relative errors and the contact points (connecting segments), the values of the respective coefficients k_0 and m_R were calculated using the serial approximation method described in [33]. As a result we obtain the following values:

$$\begin{aligned}m_R &= 0.4641016747423993624451768, R_{1\min} = 0x1fbb67af, \\ k_0 &= 0.49984979429967059464765.\end{aligned}$$

The magic constant remained the same as previously, but a significant reduction of the error can be obtained by modifying the Heron formula. The code of the corresponding Algorithm 3 with two iterations is as follows.

Algorithm 3 `SQRT_Modified_Heron_2`.

```
float SQRT_Modified_Heron_2(float x)
{ float y ;
  int i = *(int*) &x ;
  i = 0x1fbb67af + (i >> 1) ;
  y = *(float*) &i ;
  y = 0.499849794299670f * (y + x/y) ;
  y = 0.5f * (y + x/y) ;
  return y ; }
```

For this algorithm, the MREs in the first and second iterations are
 $\delta_{1\min} = -0.3003e - 3$; $\delta_{1\max} = 0.3003e - 3$; (with 11.70 correct bits)
 $\delta_{2\min} = -1.1917e - 7$; $\delta_{2\max} = 1.1921e - 7$; (with 23.00 correct) bits)

Charts of the relative errors $y \cdot 10^{-4}$ after the first iteration of algorithms SQRT_Heron-Muller and SQRT_Modified_Heron_2, for arguments ranging over $[1, 16]$, are depicted in Fig. 2.

As shown in Fig. 2, the second modified algorithm guarantees approximately half of the MREs, which are roughly symmetrical with respect to zero. It can be theoretically proven that for the classical Heron formula (1), the relation between the MRE of the initial approximation $\delta_{0\max}$ and the MRE for the n th iteration $\delta_{n\max}$ is described by

$$\delta_{n\max} = \frac{\delta_{0\max}^{2^n}}{2^{2^n} - 1};$$

and for our formula (4) with the same initial approximation,

$$\delta_{n\max} = \frac{\delta_{0\max}^{2^n}}{2^{2^{n+1}-2}};$$

For $n = 1$, the error is reduced by one-half, for $n = 2$, to one-eighth, for $n = 3$, by a factor of 128. However, in practice, such a reduction cannot always be achieved due to the limited number of bits of the mantissa and the impossibility of establishing the exact theoretical value of the coefficient k_n from formula (4) at the last iteration (therefore, it often has to be given in the rounded form of 0.5). However, the transition to a format with a larger number of bits in the mantissa (for example, from single to double precision) confirms the correctness of the theory. Below, we present the code for the same algorithm but for double precision, referred to as SQRT_Modified_Heron_2d (Algorithm 4).

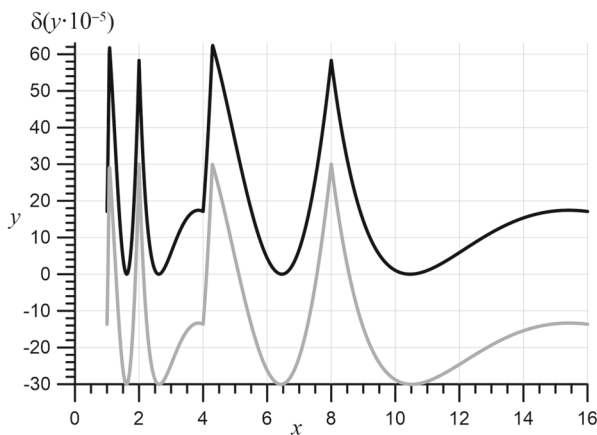


Fig. 2 The relative errors of the algorithms SQRT_Heron-Muller (in black) vs. SQRT_Modified_Heron_2 (in gray) after the second iteration: $\delta = y(x) \cdot 10^{-5}$, for $x \in [1, 16]$

Algorithm 4 Sqrt_Modified_Heron_2d.

```
double Sqrt_Modified_Heron_2d(double x)
{ union { double f; uint64_t u; } y = { x };
  y.u = 0x1ff76cf5e0b09955 + (y.u >> 1);
  y.f = 0.49984979429967059465 * (y.f + x/y.f);
  y.f = 0.49999998871785762794 * (y.f + x/y.f);
  y.f = 0.5 * (y.f + x/y.f);
  return y.f; }
```

When we compare the errors of our algorithm and the algorithm with three classical Heron iterations (assuming that Sqrt_Modified_Heron_2d and Sqrt_Classical_Heron_2d run at the same speed), the gain in precision is noticeable:

$$\delta_{1\min} = -0.3004e - 3; \delta_{1\max} = 0.3004e - 4; \text{ (vs. } \delta_{1\max} = 6.010e - 4; \text{)}$$

$$\delta_{2\min} = -2.255e - 8; \delta_{2\max} = 2.255e - 8; \text{ (vs. } \delta_{2\max} = 1.805e - 7; \text{)}$$

$$\delta_{3\min} = -2.221e - 16; \delta_{3\max} = 4.441e - 16; \text{ (vs. } \delta_{3\max} = 1.643e - 14; \text{)}$$

After the first iteration, the MRE is reduced to one-half, after the second, to one-eighth (see Fig. 3), after the third, by a factor of 36. A reduction by a factor of 128 in the MRE can be achieved in the third iteration if we switch to quadruple precision with the coefficient $k_3 = 0.49999999999999993590$.

In our paper, we do not use algorithms like Halley or Householder [5, 11], because they are more complicated and slower when compared to the Heron formula due to the additional operations in the denominator and numerator (Halley). We try to improve the initial approximation so as to reduce the number of Heron iterations. Such an approach to designing algorithms for the square root function consists of the following steps. The whole range $[1, 4)$ is divided into two segments $[1, 2)$ and $[2, 4)$ (cf. [1, 10, 30]). Then, for each segment, we find magic constants which minimize the MREs in the first iteration. Simultaneously, there is a need for one more multiplication operation in order to ensure minimal values of the errors of the piecewise linear initial approximations obtained by means of the magic constants – the corresponding code of Algorithm 5 is called Sqrt_Heron_2-segments.

Algorithm 5 Sqrt_Heron_2-segments.

```
float Sqrt_Heron_2-segments(float x)
{ int i = *(int*) & x;
  int j = (i & 0x00800000);
  if k == 0 then
    { i = 0x1f9a827a + (i >> 1);
      k0 = 1.18035706419541798f; }
  else
    { i = 0x1f5a827a + (i >> 1);
      k0 = 1.6692769686280501095f; }
  end
  float y = *(float*) & i;
  y = k0[j] * y;
  y = 0.4999930253152879213f * (y + x/y);
  return y; }
```

The MREs of Sqrt_Heron_2-segments are

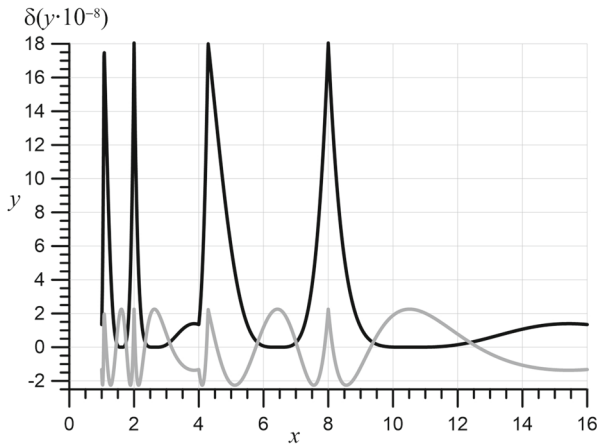


Fig. 3 The relative errors of the codes SQRT_Classical_Heron_2d (in black) vs. SQRT_Modified_Heron_2d (in gray) after the second iteration: $\delta = y(x) \cdot 10^{-8}$, for $x \in [1, 16]$

$$\delta_{\min} = -1.41e - 5; \delta_{\max} = 1.41e - 5; \text{ (with 16.11 correct bits)}$$

Apparently, only one iteration of SQRT_Heron_2-segments is sufficient to decrease the relative errors by a factor of approximately 21 when compared to the first iteration of SQRT_Modified_Heron_2, and by a factor of 42 from the first iteration of SQRT_Heron-Muller. If we add one more classical Heron iteration, then we get an accuracy of 23 bits (for *float*) and 33 bits (for *double*).

If the idea of further partitioning of segments into subsegments is implemented, we can significantly increase the accuracy of the square root approximation. For instance, we can divide segments $[1, 2)$ and $[2, 4)$ into the same even number of subsegments. Our solution comes from partitioning the first mantissa segment into 2^k , $k \in \{1, 2, 3, 4, 5, \dots\}$, equal subsegments $p : [a, b)$, where $a = 2^{(j-1)}$, $b = 2^j$, for $p \in \{2, 4, 8, 16, 32, \dots\}$, and $j = \{1, \dots, p\}$. The second segment $[2, 4)$ is divided similarly, but the ends of the ranges $p : [a, b)$ are $a = 2 * 2^{(j-1)}$, $b = 2 * 2^j$. Next, for all created subsegments, magic constants must be found.

Algorithm 6 with sixteen subsegments is presented below. The MREs of SQRT_16-segments are

$$\delta_{\min} = -2.20e - 4; \delta_{\max} = 2.20e - 4; \text{ (with 12.15 correct bits).}$$

It should be noted that the code of SQRT_16-segments is straightforward, fast, and provides an accuracy of about 12.15 correct bits. Such an accuracy (with a minimal hardware cost, a single FP multiplier) is sufficient for many practical applications, e.g. accelerators for neural network tasks, power electronics, machine learning, computer vision and intelligent robotic systems [1, 34, 42]. From our research, we know that for two segments, we can obtain 7 correct bits, for four subsegments, 8.6 correct bits, and for eight subsegments, 10 correct bits of the result. In order to increase the accuracy of the approximation, we can increase the number of subsegments. For instance, 32 subsegments achieves 14-bit accuracy, 64 achieves 16-bit, 128, 18-bit, 256, 20-bit, 512, 22-bit, and 1024, 24-bit accuracy. This approach can be adapted to different formats with small mantissa lengths (equally fast algorithms for the square root but with lower

accuracy are described in [23]): bfloat, 7 bits, half, 10 bits. The disadvantage of this approach is the size of the look-up table to store the values of $k0[\]$ and $R[\]$, especially for devices with limited memory resources such as microcontrollers, for which another type of algorithm must be implemented.

After adding a single iteration of the classical Heron formula, SQRT_16-segments obtains the following relative errors (its code is available from our GitHub repository):

$$\delta_{min} = -8.8615e - 8; \delta_{max} = 1.1080e - 7; \text{ (with 23.0 correct bits).}$$

It is worth noticing that this code has potentially sufficient accuracy (on the level of 24 correct bits), which in the FP *float* format used here is limited by rounding errors.

In this case, when using single precision, there is no need to change the value of the coefficient 0.5 in the Heron formula. As results from the above characteristics, SQRT_Heron_16-segments ensures a sufficiently high accuracy of approximation of the square root, incurring relatively low computational complexity. However, due to their complexity and considerable duration, the division operations should be as a rule avoided when implementing square root algorithms on FPGAs. In this case, we used the above mentioned multiplication-based Newton–Raphson formula (2) [1, 24, 26, 31, 34, 36, 38, 42] for the inverse square root, which does not contain division operations.

It should be noted that direct hardware implementation of inverse square root algorithms on FP blocks requires a large amount resources, i.e. three or four FP multipliers and one FP adder per iteration [24, 36, 42]. It is precisely due to the complexity of the Newton–Raphson iterations for the inverse square root, that such software algorithms have no advantages over the algorithms that use Heron iterations (see test results in Section 6). We have to note here that modern high-speed CPUs (Intel, AMD), IPUs, microcontrollers (ARM and STM), and single-board computers (SBCs) have hard-

Algorithm 6 SQRT_16-segments.

```
const float k0[16] =
{ 1.3728804844963743741743109895869174,      1.2984464325688431991846789745811575,
  1.2349522866656504095676954218143650,      1.1799541552589359812272435026388901,
  1.1317105556304312508949849916223526,      1.0889423218359534849831706863530610,
  1.0506858108072537800493465016626868,      1.0161993590210952468964805394085166,
  1.9415462006921182911486340105462846,      1.8362805549538204860640915291539554,
  1.7464862726862291524227673452809011,      1.6687071693456759394814332790086739,
  1.6004804164533469399066344222701610,      1.5399970001824531463719726207375215,
  1.4858941234365901375799250869156190,      1.4371229156024788917009615207303487};
const int R[16] =
{ 0x1f83e1dc, 0x1f8be509, 0x1f93e79b, 0x1f9be9bb, 0x1fa3eb84, 0x1fabad08, 0x1fb3ee57, 0x1fbbe7b,
  0x1f43e1dc, 0x1f4be509, 0x1f53e79b, 0x1f5be9bb, 0x1f63eb84, 0x1f6bed08, 0x1f73ee57, 0x1f7bef7b};
float SQRT_16-segments(float x)
{ float y; uint32_t i, j, i_mx;
  i = (int*) & x;
  i_mx = i & 0x00ffff;
  j = i_mx >> 20;
  i = R[j] + (i >> 1);
  y = *(float*) & i;
  y = k0[j] * y;
  return y; }
```

ware FP instructions for the (reciprocal) square root (which provide an accuracy of 8, 11, 14, 23, or 28 bits), which allows $rsqrt/sqrt$ to be calculated quickly. For instance, the Raspberry Pi SBC offers a rough initial approximation of the reciprocal square root with 8.25 correct bits and $\delta_{\max} = 3.28e - 3$, and multiplication-based Newton–Raphson iterations implemented in hardware.

However, most platforms with limited hardware resources (such as low-cost ESP32 microcontrollers) can at most have hardware FP adders, multipliers and FMAs (fused multiple-add). In this case, they can only rely on algorithms from the C library, which are slow [18, 34], or algorithms from commercial libraries, which are fast but expensive.

5 Other high-quality algorithms with a specific accuracy

Below we present the additional code for four such algorithms reworked from [34, 35], which compute the square root with varying degrees of accuracy and do not need any division operations. They are favourable compared with other algorithms reported in the literature and developed for the same accuracy.

The code of the new algorithms' with 10-bit, 13-bit, 15-bit and 23-bit accuracy are presented below.

Algorithm 7 SQRT_10-bits.

```
float SQRT_10-bits(float x)
{int i = *(int*) & x ;
 i = 0x5f0b3892 - (i >> 1) ;
 float y = *(float*) & i ;
 float c = x * y ;
 y = c * (1.89099014875f - c * y) ;
 return y ;}
```

The MREs of the algorithm SQRT_10-bits are

$\delta_{\min} = -8.802e - 4$; $\delta_{\max} = 8.802e - 4$; (with 10.14 correct bits).

In comparison to the well-known Quake Arena 3 algorithm [29], our Algorithm 7 has three multiplications vs four, and the MRE almost one-half ($8.802e - 4$ vs $1.751e - 3$).

Comparing Algorithm 8 (SQRT_13-bits) to the code of the algorithm from [28], we have four multiplications vs five, and 1.34 times smaller MRE ($7.470e - 5$ vs. $1.007e - 4$).

Another algorithm for the required 15-bit accuracy is proposed.

Comparing Algorithm 9 (SQRT_15-bits) to the algorithm by [28], our algorithm needs only four multiplications vs five, and its MRE $2.027e - 5$ is 4.96 times smaller than $1.007e - 4$.

Let's take a look at our Algorithm 10, which provides 23-bit accuracy. The algorithm SQRT_23-bits needs only seven multiplications vs the ten required in [36, 41].

Algorithm 8 Sqrt_13-bits.

```

const float k0[2] = {0.88601269f, 1.25302145f} ;
const float k1[2] = {0.103027083f, 0.291411832f} ;
const int R[2] = {0x5f99e8b6, 0x5f59e8b6} ;
float Sqrt_13bits(float x)
{
  float y, c ;
  { int i = *(int*) & x ;
    int j = (i & 0x00800000) >> 23 ;
    i = R[j] - (i >> 1) ;
    y = *(float*) & i ;
    c = x * y ;
    y = c * (k0[j] - k1[j] * c * y) ;
    return y ; }
}

```

Algorithm 9 Sqrt_15-bits.

```

float Sqrt_15-bits(float x)
{
  int i = *(int*) & x ;
  i = 0x5f1110a0 - (i >> 1) ;
  float y = *(float*) & i ;
  float c = x * y ;
  float d = c * y ;
  y = c * (2.2825186f - d * (2.2533049f - d)) ;
  return y ; }

```

Algorithm 10 Sqrt_23-bits.

```

float Sqrt_23-bits(float x)
{
  union { float f; uint32_t u; } y = {x}, yy = {x} ;
  y.u = 0x5f1110a0 - (y.u >> 1) ;
  float c = x * y.f * y.f ;
  y.f = y.f * (2.2825186f - c * (2.2533049f - c)) ;
  yy.u = y.u - 0x00800000 ;
  y.f = x * y.f ;
  y.f = y.f + (x - y.f * y.f) * yy.f ;
  return y.f ; }

```

In conclusion, our quality algorithms with specific accuracy outperform their counterparts known from the literature in terms of the complexity of the operations and also the MRE.

6 Experimental verification and test results

In the experimental part of our work, we examined how the proposed algorithms perform and what level of improvement and accuracy they provide compared to the algorithms based on the $\text{sqrt}(x)$ function selected from publicly available and widely used libraries. We have inspected implementations of $\text{sqrt}(x)$ created by Sun Microsystems, Inc. and that taken from the Fresh Open-Source Software Archive as a component of the package *musl* (an implementation of the C standard library built on top of the Linux system).

For purposes of illustration, we have also used the results obtained from the C/C++ `<math.h>` or `<cmath>` libraries, used as a reference, knowing that the $\text{sqrt}(x)$ in all tested platforms has been implemented directly in the hardware. It is important to acknowledge that the software-based versions of algorithms do not have a chance of competing with hardware implementations.

We also have looked at the performance improvement and the level of accuracy of our implementations of $\text{sqrt}(x)$ versus the implementation of Heron in the SQRT_Heron-Muller algorithm, a fundamental point of reference, proposed in [39].

In our test scenario, we looked at various architectures, such as AMD and Intel’s x86-based superscalar processors. We also examined the 64-bit versions of ARM processors. For the SIMD and the vectorized version of the accelerator, we tested a system based on the Nvidia A-100 GPU. Graphcore IPU GS200 represents the MIMD architecture. We took all these various systems based on different architectures and conducted experiments to check whether the proposed algorithms were architecture-independent, regardless of the class in Flynn’s taxonomy. Descriptions of the tested architectures are summarized in Table 1.

In our experiments, we analysed the runtime, the number of clock cycles, RMSD, the number of correct bits calculated, the MRE (Maximum Relative Error), and the Minimal Relative Error.

The results of these computational experiments performed on five different platforms are shown further down. In all cases we used the latest available version of the compilers and SDK for the given architecture, and compilation with the -O3 maximum optimization flag. In addition to the algorithms described in previous sections, we gathered and compared the $\text{sqrt}(x)$ functions implemented in the standard C library `<math.h>` as `std::sqrtf`, as well as implementations acquired from the Fresh Open-Source Software Archive, such as `sqrt_alt` and `sqrtf_alt`. We also make a comparison

Table 1 Configurations of all tested platforms

	Intel Core i5 1038NG7	AMD EPYC 7742	ARM Marvell ThunderX2	Nvidia A-100	Graphcore GS200
Chip speed [MHz]	2000	3400	2500	765	1325
L1 cache	32 KB	32 KB	32 KB	192 KB	NA
L2 cache	256 KB	512 KB	256 KB	40960 KB	NA
L3 cache	6 MB	16384 KB	32768 KB	NA	NA
RAM	16 GB	768 GB	502 GB	40 GB	900 MB
OS	macOS	Ubuntu	Ubuntu	Ubuntu	Ubuntu
Version	12.4	18.04.4 LTS	18.04.4 LTS	18.04.4 LTS	18.04.4 LTS
C++	Apple clang	Clang 6.0.0-1	Clang 6.0.0-1	CUDA	Poplar SDK
Compiler	13.1.6	Ubuntu	Ubuntu	11.7	2.4

with the implementation of $\text{sqrt}(x)$ developed by Sun Microsystems, referred to as `sun_ieee754_sqrt`.

Table 2 shows the execution time T_{avg} for combinations of five different architectures and all the tested algorithms. This is the average time of algorithm execution, in the FP argument range $x \in [1, 4)$, obtained using *nexafterf* C++ instructions, divided by the number of iterations.

Let N_{total} denote the total number of iterations performed by the algorithm. Thus,

$$T_{avg} = \frac{\sum_{i=1}^{N_{total}} T(i)}{N_{total}},$$

where $T(i)$ denotes the execution time for the i th iteration.

Even though the time of execution is not the best metric to measure the performance of the proposed algorithms, it clearly shows that all of the algorithms execute within the range of 2–7 ns for Intel and AMD processors and 4–16 ns for ARM. In contrast, `sqrt_alt`, `sqrtf_alt` and `sun_ieee754_sqrt` require 187 ns, 120 ns, and 493 ns, respectively, on an ARM processor, 32 ns, 20 ns and 246 ns on an Intel processor, and 47 ns, 40 ns and 326 ns on an AMD processor. This is from 4–5 times up to 100 times

Table 2 Algorithms' average execution time T_{avg} for all tested platforms (in ns)

Algorithms	Intel	AMD	ARM	Nvidia	Graphcore
SQRT_Blinn	3	5	7	11329	344
SQRT_Blinn_Modified	2	2	4	10987	348
SQRT_Heron-Muller	5	5	13	12003	412
SQRT_Modified_Heron_1	4	5	11	11329	1412
SQRT_Modified_Heron_2	4	5	11	11313	2416
SQRT_2-segments	3	3	8	11329	398
SQRT_Heron_2-segments	4	5	11	12227	416
SQRT_16-segments	3	4	9	11326	362
SQRT_Heron_16-segments	4	6	13	12912	393
SQRT_10-bits	4	4	13	11189	371
SQRT_13-bits	5	6	16	11343	398
SQRT_15-bits	5	5	15	11329	375
SQRT_23-bits	7	7	16	11955	366
<code>sqrt_alt</code>	32	47	187	12138	2617
<code>sqrtf_alt</code>	20	40	120	11955	846
<code>sun_ieee754_sqrt</code>	246	326	493	13864	5055
<code>std::sqrtf</code>	1	2	5	11513	4
<code>std::sqrt</code>	1	4	6	11513	

slower than our proposed algorithms. Of course, when we compare our results to the hardware implementation of $\text{sqrt}(x)$ measured by `std::sqrtf` and `std::sqrt`, we see that our implementations made in software are 2–7 times slower.

In contrast to CPUs, the execution time on a GPU or IPU is significantly affected by the communication between the host and the GPU/IPU accelerators, so it doesn't make much sense to compare the execution time in the offload accelerator scenario versus CPUs. For the Nvidia A-100 and Graphcore GS200 chips, the execution times are significantly longer than the CPU-based results. Interestingly, in the case of the Nvidia A-100 GPU based solution, we see only a 21% difference between all the tested algorithms, regardless of whether it was a software or hardware version of the implementation. For the Graphcore GS200 based implementation, we observe a very effective hardware implementation of $\text{sqrt}(x)$ called up by `std::sqrtf` that needs only 4 ns to execute, in contrast to 5055 ns, 2617 ns, and 846 ns for `sun_ieee754_sqrt`, `sqrt_alt`, and `sqrtf_alt`, respectively.

The number of cycles is a much better indicator of performance than execution time, but we need to look at the results achieved in correlation with the RMSD, the number of correct bits, or with the MRE.

Table 3 shows the number of clock cycles for all the evaluated algorithms running on five different platforms. The CPU cycles were measured using `__builtin_readcycle`

Table 3 The number of clock cycles for all tested platforms

Algorithms	Intel	AMD	ARM	Nvidia	Graphcore
SQRT_Blinn	4.41	5.59	0.85	365	456
SQRT_Blinn_Modified	4.23	5.59	0.72	667	462
SQRT_Heron-Muller	9.12	12.20	2.63	885	546
SQRT_Modified_Heron_1	8.72	11.22	2.61	884	1546
SQRT_Modified_Heron_2	8.91	11.36	2.57	871	2552
SQRT_2-segments	7.08	8.31	2.21	884	528
SQRT_Heron_2-segments	9.00	11.56	2.32	872	552
SQRT_16-segments	6.69	11.15	2.11	881	480
SQRT_Heron_16-segments	9.46	11.61	2.64	888	522
SQRT_10-bits	9.34	10.23	2.05	628	492
SQRT_13-bits	11.35	13.52	2.91	836	528
SQRT_15-bits	10.56	12.39	3.39	886	498
SQRT_23-bits	13.96	17.76	5.12	894	486
<code>sqrt_alt</code>	66.66	106.04	37.45	1138	3468
<code>sqrtf_alt</code>	41.28	90.93	37.47	944	1122
<code>sun_ieee754_sqrt</code>	498.52	731.37	98.73	3267	6699
<code>std::sqrtf</code>	3.22	6.62	1.04	502	30
<code>std::sqrt</code>	3.11	9.97	1.28	502	

counter(). This function is a compiler intrinsic that reads the CPU cycle counter, providing a low-level interface to measure the performance on a given architecture.

We can see that the most efficient algorithm is a hardware implementation called up by `std::sqrtf`, but also the simple rough square root approximations of `SQRT_Blinn` and `SQRT_Blinn_Modified` are almost equally efficient. For the ARM and Nvidia A-100 GPU this simple rough square root approximation uses the smallest number of cycles. If we look at the corresponding RMSD or the number of correct bits we see huge discrepancies: `std::sqrtf` guarantees a reference precision and accuracy around 24 bits and `SQRT_Blinn` achieves only 4.04 bits (cf. Tables 4 and 5).

When we look more deeply at all our algorithms and analyse the cycles which are characteristic for them, we clearly see that the fastest are `SQRT_10-bits` and `SQRT_2-segments`. If the precision of the calculation is not our main concern and 7–10 correct bits is a good level of accuracy, then the fastest implementations of the $\sqrt{x}$ are `SQRT_10-bits` and `SQRT_2-segments`, which provide 9–70 times faster execution time than `sqrt_alt`, and `sqrtf_alt` `sun_ieee754_sqrt`. Of course, we need to take into account that the achieved precision is just 7–10 correct bits versus the 24 bits guaranteed by the software version of `sqrt_alt`, `sqrtf_alt` and `sun_ieee754_sqrt`.

If we compare `SQRT_Modified_Heron_1` with the `SQRT_Heron-Muller` reference implementation, we see a slightly smaller number of clock cycles for our algorithms utilizing Intel, AMD, ARM, NVIDIA with the same precision achieved by both

Table 4 The number of correct bits for CPUs, Nvidia A-100 GPU, Graphcore GS200 IPU

Algorithms	Intel/AMD/ARM	Nvidia	Graphcore
<code>SQRT_Blinn</code>	4.04	4.04	4.72
<code>SQRT_Blinn_Modified</code>	4.85	4.85	5.28
<code>SQRT_Heron-Muller</code>	21.90	23.00	23.45
<code>SQRT_Modified_Heron_1</code>	21.91	22.87	23.46
<code>SQRT_Modified_Heron_2</code>	23.00	23.00	23.79
<code>SQRT_2-segments</code>	7.06	7.06	7.05
<code>SQRT_Heron_2-segments</code>	16.11	16.12	16.07
<code>SQRT_16-segments</code>	12.15	12.17	12.99
<code>SQRT_Heron_16-segments</code>	23.00	23.00	24.00
<code>SQRT_10-bits</code>	10.15	10.15	9.89
<code>SQRT_13-bits</code>	13.71	13.71	13.70
<code>SQRT_15-bits</code>	15.59	15.59	5.83
<code>SQRT_23-bits</code>	23.00	22.78	22.78
<code>sqrt_alt</code>	24.00	inf	inf
<code>sqrtf_alt</code>	inf	inf	inf
<code>sun_ieee754_sqrt</code>	24.00	inf	inf
<code>std::sqrtf</code>	inf	inf	inf
<code>std::sqrt</code>	inf	inf	

Table 5 RMSD for CPUs, Nvidia A-100 GPU and Graphcore GS200 IPU

Algorithms	Intel/AMD/ARM	Nvidia	Graphcore
SQRT_Blinn	3.836E-02	3.787E-02	3.787E-02
SQRT_Blinn_Modified	2.643E-02	2.570E-02	2.569E-02
SQRT_Heron-Muller	8.733E-08	8.733E-08	8.733E-08
SQRT_Modified_Heron_1	8.690E-08	8.690E-08	8.690E-08
SQRT_Modified_Heron_2	6.873E-08	6.873E-08	6.873E-08
SQRT_2-segments	7.543E-03	7.522E-03	7.522E-03
SQRT_Heron_2-segments	1.466E-05	1.454E-05	1.454E-05
SQRT_16-segments	1.249E-04	1.233E-04	1.233E-04
SQRT_Heron_16-segments	5.964E-08	5.964E-08	5.964E-08
SQRT_10-bits	1.062E-03	1.050E-03	1.050E-03
SQRT_13-bits	7.593E-05	7.529E-05	7.529E-05
SQRT_15-bits	1.734E-05	1.716E-05	1.716E-05
SQRT_23-bits	6.276E-08	6.276E-08	6.276E-08
sqrt_alt	3.441E-08	3.599E-08	0.000E+00
sqrtf_alt	0.000E+00	0.000E+00	0.000E+00
sun_ieee754_sqrt	3.441E-08	3.599E-08	0.000E+00
std::sqrtf	0.000E+00	0.000E+00	0.000E+00
std::sqrt	0.000E+00	0.000E+00	0.000E+00

algorithms. In order to increase the precision, we can use SQRT_Modified_Heron_2 algorithm with a modification of the coefficient k_0 in the first iteration of the Heron formula. This modification provides 23.0 correct bits (versus 21.91 bits) and the number of cycles is still smaller than for SQRT_Heron-Muller.

Table 4 shows the number of correct bits for all algorithms. The highest precision is achieved by SQRT_Modified_Heron_2 and SQRT_Heron_16-segments, which yield 23.00 correct bits for all tested platforms. The higher level of precision, 24 bits, is only achieved by the reference libraries-based implementations sqrt_alt, sqrtf_alt, sun_ieee754_sqrt and their hardware versions std::sqrtf and std::sqrt. Table 5 presents the RMSD for all five platforms. Intel, AMD, ARM achieve the same values of the RMSD, but the Nvidia A-100 and Graphcore GS200 chips have different RMSDs. The lowest RMSD of all the algorithms we implemented was for SQRT_Heron_16-segments. This proves the validity of our idea to split the x range into subsegments in order to improve the accuracy over the entire calculated range. The achieved result is more than 32% better than the reference implementation of SQRT_Heron-Muller, not taking into account the functions implemented using the reference libraries, sqrtf_alt, std::sqrtf and sun_ieee754_sqrt. If we prefer making the comparison by using the Maximum Relative Error as the indicator for the best precision for our algorithms, then SQRT_Modified_Heron_2 also achieves 50% better accuracy than our other algorithms as well as versus SQRT_Heron-Muller – excluding the functions implemented

in the reference libraries, `sqrtf_alt`, `std::sqrtf`, and `sun_ieee754_sqrt`. The comparison is shown in Table 6.

Table 7 presents the Minimum Relative Errors for all tested algorithms and validated on all platforms. The discrepancy in this case is minimal and all values are almost identical on all platforms, nor is there much difference between the different types of algorithms.

7 Conclusions

In this paper, a new approach to the application of Newton–Raphson (Heron) formula for computing square roots has been proposed. In particular, the use of magic constants is recommended for finding the initial approximation for Heron iterations. Such a technique ensures the fast creation of sufficiently accurate initial values for the computation of the square root due to a partitioning of range of arguments into segments, which are further partitioned into an uneven number of subsegments. In addition, we have proposed a modification of the Heron formula in a way that reduces the errors of the approximation after each iteration, at the cost of only two additional instructions: one multiplication and one conditional instruction *if-else*. We have also proposed simple modifications to the Newton–Raphson formula and the magic number method suitable for implementation on platforms with limited hardware resources, such as microcontrollers, FPGAs etc.

Table 6 Maximum relative error for CPUs, Nvidia A-100 GPU, Graphcore GS200 IPU

Algorithms	Intel/AMD/ARM	Nvidia	Graphcore
SQRT_Blinn	6.0660E-02	6.0660E-02	6.0660E-02
SQRT_Blinn_Modified	3.4747E-02	3.4765E-02	3.4747E-02
SQRT_Heron-Muller	2.5568E-07	2.3823E-07	2.3842E-07
SQRT_Modified_Heron_1	2.5343E-07	2.3845E-07	2.3842E-07
SQRT_Modified_Heron_2	1.1921E-07	1.1901E-07	1.1921E-07
SQRT_2-segments	7.4978E-03	7.4979E-03	7.4979E-03
SQRT_Heron_2-segments	1.4102E-05	1.4047E-05	1.4067E-05
SQRT_16-segments	2.1692E-04	2.1696E-04	2.1696E-04
SQRT_Heron_16-segments	1.1921E-07	1.1921E-07	1.1921E-07
SQRT_10-bits	8.8018E-04	8.8012E-04	8.8012E-04
SQRT_13-bits	7.4582E-05	7.4625E-05	7.4625E-05
SQRT_15-bits	2.0239E-05	2.0266E-05	2.0266E-05
SQRT_23-bits	1.19007E-07	1.3842E-07	1.3842E-07
<code>sqrt_alt</code>	5.9605E-08	0.0000E+00	0.0000E+00
<code>sqrtf_alt</code>	0.0000E+00	0.0000E+00	0.0000E+00
<code>sun_ieee754_sqrt</code>	5.9605E-08	0.0000E+00	0.0000E+00
<code>std::sqrtf</code>	0.0000E+00	0.0000E+00	0.0000E+00
<code>std::sqrt</code>	0.0000E+00	0.0000E+00	

Table 7 Minimum relative error for CPUs, Nvidia A-100 GPU, Graphcore GS200 IPU

Algorithms	Intel/AMD/ARM	Nvidia	Graphcore
SQRT_Blinn	0.0000E+00	0.0000E+00	0.0000E+00
SQRT_Blinn_Modified	−3.4747E−02	−3.4747E−02	−3.4747E−02
SQRT_Heron-Muller	−1.1917E−07	−1.1921E−07	−1.1921E−07
SQRT_Modified_Heron_1	−1.1917E−07	−1.1921E−07	−1.1921E−07
SQRT_Modified_Heron_2	−1.1918E−07	−1.1921E−07	−1.1921E−07
SQRT_2-segments	−7.4421E−03	−7.4421E−03	−7.4421E−03
SQRT_Heron_2-segments	−1.4080E−05	−1.4067E−05	−1.4067E−05
SQRT_16-segments	−2.1681E−04	−1.4067E−05	−2.1678E−04
SQRT_Heron_16-segments	−1.1917E−07	−1.1921E−07	−1.1921E−07
SQRT_10-bits	−8.8018E−04	−8.8018E−04	−8.8018E−04
SQRT_13-bits	−7.4675E−05	−7.4685E−05	−7.4685E−05
SQRT_15-bits	−2.0271E−05	−2.0266E−05	−2.0266E−05
SQRT_23-bits	−1.18762E−07	−1.17883E−07	−1.17881E−07
<i>sqr</i> _alt	−5.9565E−08	0.0000E+00	0.0000E+00
<i>sqr</i> t_f_alt	0.0000E+00	0.0000E+00	0.0000E+00
<i>sun_ieee754_sqrt</i>	−5.9565E−08	0.0000E+00	0.0000E+00
<i>std::sqrtf</i>	0.0000E+00	0.0000E+00	0.0000E+00
<i>std::sqrt</i>	0.0000E+00	0.0000E+00	0.0000E+00

In the section about the verification of the performance of these proposals, we focused on the validation of our algorithms against the known software and hardware implementations of $\text{sqrt}(x)$ available from standard C and C++ libraries as well as created by Sun Microsystems and an implementation accessible as a member of the package *musl*. We also verified the performance and accuracy of our versions of Heron's algorithm, which were tested against the reference SQRT_Heron-Muller algorithm. We validated our results on different types of architecture and platforms to demonstrate the versatility of our algorithms and their platform agnosticism. We compared the algorithms on AMD, ARM, and Intel CPUs, Nvidia A-100 GPU and Graphcore IPU platforms. The results clearly show that the proposed approaches and modifications of the Heron algorithm provide sufficient performance characteristics and significantly outperform the $\text{sqrt}(x)$ algorithms implemented in the package *musl* of the Fresh Open-Source Software Archive as well as the version of the $\text{sqrt}(x)$ function created by Sun Microsystems. Our proposed Heron-based algorithms outperform the publicly known and available software implementations of the function $\text{sqrt}(x)$ as well as the classical implementation of the Heron algorithm proposed by [39], also providing higher accuracy. They are also good alternatives for hardware implementations.

In conclusion, our contribution to the approximation of the square root is the next step towards building an open library of fast algorithms for computing popular mathematical functions to meet the expectations of scientists and engineers who conduct numerical computations in signal processing and other demanding applications on dedicated hardware platforms.

Acknowledgements This paper and the research behind it would not have been possible without the exceptional support of Graphcore Customer Engineering and Software Engineering team. We would like to express our appreciation to Hubert Chrzaniuk, Krzysztof Góreczny, Grzegorz Andrejczuk and Bartłomiej Kocot for their valuable and constructive suggestions connected to the testing of our algorithms and development of this research work.

For computations we used PLGrid infrastructure at ACK Cyfronet AGH, Krakow, Poland.

Author Contributions Z.K., V.S., and N.G. : project administration, methodology, analysis of results, preparation of writing, figures - review & editing; P.G. and L.M. : conceptualisation, methodology, software, programming, writing the original draft. M.W. : software, validation, visualization. [All authors reviewed the manuscript.] All authors contributed equally to the reported research.

Funding No funding was received for conducting this study.

Data and Code Availability The code examples and data that support the findings of this study are openly available in GitHub repository: <https://github.com/pawelgepner/Heron-Square-Root-Function>.

Declarations

The authors have no relevant financial or non-financial interests to disclose.

There is no approval committee for our research and the submitted article.

Competing Interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Altamimi, A., Youssef, B.B.: Novel seed generation and quadrature-based square rooting algorithms. *Sci. Rep.* **12**(1), 20540 (2022). <https://doi.org/10.1038/s41598-022-25039-y>
- Andrews, M.: Mathematical microprocessor software: a $\sqrt{x}$ comparison. *IEEE Micro* **2**(2), 63–79 (1982). <https://doi.org/10.1109/MM.1982.290970>
- Anghel, C., Paleologu, C., Benesty, J., Ciochină, S.: FPGA implementation of a variable step-size affine projection algorithm for acoustic echo cancellation. In: 18th European Signal Processing Conference (EUSIPCO-2010), Aalborg, Denmark, 23–27 August 2010, pp. 532–536 (2010). <https://doi.org/10.5281/zenodo.41864>
- Anghel, C., Ciochina, S.: On the FPGA implementation of the VR-RLS algorithms. In: The Sixteenth International Conference on Networks (ICN 2017), Venice, Italy, 23–27 April, 2017, pp. 98–101. Available online: <https://api.semanticscholar.org/CorpusID:250447098> (2017)
- Beebe, N.H.F.: The mathematical-function computation handbook: programming using the mathCW portable software library. Springer-Verlag: Berlin, pp. 215–242 (Roots) (2017). <https://doi.org/10.1007/978-3-319-64110-2>
- Blinn, J.F.: Floating-point tricks. *IEEE Comput. Graph. Appl.* **17**(4), 80–84 (1997). <https://doi.org/10.1109/38.595279>
- Boldo, S., Jeannerod, C.-P., Melquiond, G., Muller, J.M.: Floating-point arithmetic. *Acta Numer.* **32**, 203–290 (2023). <https://doi.org/10.1017/S0962492922000101>

8. Bruguera, J.D.: Low latency floating-point division and square root unit. *IEEE Trans. Comput.* **69**(2), 274–287 (2020). <https://doi.org/10.1109/TC.2019.2947899>
9. Chen, J., Xue, L., Anderson J.H.: Software-specified FPGA accelerators for elementary functions. In: 2018 International conference on Field-Programmable Technology (FPT), Naha, Japan, 10–14 December, pp. 54–61 (2018). <https://doi.org/10.1109/FPT.2018.00019>
10. Chen, J.: Hardware acceleration for elementary functions and RISC-V processor. Ph.D. Thesis, McGill University, Montreal, QC, Canada (2020)
11. Chowdhury, R.A., Kaykobad, M.: Calculating the square root with arbitrary order of convergence. *Int. J. Comput. Math.* **75**(3), 297–302 (2006). <https://doi.org/10.1080/00207160008804985>
12. CMSIS DSP Software Library: CMSIS-DSP Version 1.10.0. Available online: <https://www.keil.com/pack/doc/CMSIS/DSP/html/index.html>
13. Crawford, J.A.: Computing square roots. AM1 LLC, U11891. Available online: <http://www.am1.us/> (2005)
14. Detmer, R.C.: Introduction to 80x86 assembly language and computer architecture, 3rd edn., pp. 99–122. Jones, Bartlett Learning (2006)
15. Dutta, S., Tavva, Y., Bhattacharjee, D., Chattopadhyay, A.: Efficient quantum circuits for square-root and inverse square-root. In: 2020 33rd International conference on VLSI design and 2020 19th International conference on embedded systems (VLSID), Bangalore, India, pp. 55–60 (2020). <https://doi.org/10.1109/VLSID49098.2020.00027>
16. van Eekelen, M., Frumin, D., Geuvers, H., Gondelman, L., Krebbers, R., Schoolderman, M., Smetsers, S., Verbeek, F., Viguier, B., Wiedijk, F.: A benchmark for C program verification. *arXiv:1904.01009* (2019)
17. Ercegovic, M.D., Lang T.: Digital Arithmetics. Chap. 7. ‘Reciprocal, Division, Reciprocal Square Root, and Square Root by Iterative Approximation’, pp. 366–395. Chap. 11. ‘Cordic algorithm and implementations’, pp. 608–648. Morgan Kaufmann (2004). <https://doi.org/10.1016/B978-155860798-9/50009-9>
18. Fog, A.: Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD, and VIA CPUs. Available online: https://www.agner.org/optimize/instruction_tables.pdf (2019)
19. Gepner, P., Gamayunov, V., Fraser L.: Effective implementation of DGEMM on modern multicore CPU. In: International Conference on Computational Science, ICCS 2012 (2012). <https://doi.org/10.1016/j.procs.2012.04.014>
20. Gepner, P., Fraser, D., Kowalik M.: Second generation quad-core Intel Xeon processors bring 45 nm technology and a new level of performance to HPC applications. In: International Conference on Computational Science, ICCS 2008 (2008). https://doi.org/10.1007/978-3-540-69384-0_47
21. Granlund, T.: Instruction latencies and throughput for AMD and Intel x86 processors. Available online: <https://gmplib.org/~tege/x86-timing.pdf> (2019)
22. Gustafsson, O., Wanhammar, L.: Square root computation. Polynomial and piecewise polynomial approximations. In: Arithmetic circuits for DSP applications. Meher, P.K., Stouraitis, T. (eds.) IEEE Press, Wiley, pp. 27–29 (2017)
23. Gustafsson, O., Hellman, N.: Approximate floating-point operations with integer units by processing in the logarithmic domain. In: 2021 IEEE 28th Symposium on computer arithmetic (ARITH), Lyngby, Denmark, 14–16 June, 2021, pp. 45–52 (2021). <https://doi.org/10.1109/ARITH51176.2021.00019>
24. Hasnat, A., Bhattacharyya, T., Dey, A., Halder, S., Bhattacharjee D.: A fast FPGA based architecture for computation of square root and inverse square root. In: 2017 Devices for Integrated Circuit (DevIC), Kalyani, India, 23–24 March, pp. 383–387 (2017). <https://doi.org/10.1109/DEVIC.2017.8073975>
25. Joldes, M., Marty, O., Muller, J., Popescu, V.: Arithmetic algorithms for extended precision using floating-point expansions. *IEEE Trans. Comput.* **65**(4), 1197–1210 (2015). <https://doi.org/10.1109/TC.2015.2441714>
26. Karp, A.H., Markstein, P.: High-precision division and square root. *ACM Trans. Math. Softw.* **23**(4), 561–589 (1997). <https://doi.org/10.1145/279232.279237>
27. Kornerup, P., Muller, J.-M.: Choosing starting values for certain Newton-Raphson iterations. *Theor. Comput. Sci.* **351**(1), 101–110 (2006). <https://doi.org/10.1016/j.tcs.2005.09.056>
28. Lemaitre, F., Couturier, B., Lacassagne, L.: Cholesky factorization on SIMD multi-core architectures. *J. Syst. Architect.* **79**, 1–15 (2017). <https://doi.org/10.1016/j.sysarc.2017.06.005>
29. Lomont, C.: Fast inverse square root. Purdue University, Tech. Rep. Available online: <http://www.lomont.org/Math/Papers/2003/InvSqrt.pdf> (2003)

30. Montuschi, P., Mezzalama, M.: Survey of square rooting algorithms. *IEE Proc. Comput. Digit. Tech.* **137**(1), 31–40 (1990). <https://doi.org/10.1049/IP-E.1990.0003>
31. Montuschi, P., Mezzalama, M.: Optimal absolute error starting values for Newton-Raphson calculation of square root. *Computing* **46**, 67–86 (1991). <https://doi.org/10.1007/BF02239012>
32. Moroz, L., Walczyk, C.J., Hrynchyshyn, A., Holimath, V., Cieřliński, J.L.: Fast calculation of inverse square root with the use of magic constant - Analytical approach. *Appl. Math. Comput.* **316**, 245–255 (2018). <https://doi.org/10.1016/j.amc.2017.08.025>
33. Moroz, L., Samoty, V., Horyachyy O., Dzelendzyak, U.: Algorithms for calculating the square root and inverse square root based on the second-order Householder's method. In: *Proceedings of the 2019 10th IEEE International conference on Intelligent Data Acquisition and Advanced Computing Systems: technology and applications (IDAACS)*, Metz, France, 18–21 September, 2019, pp. 436–442 (2019). <https://doi.org/10.1109/IDAACS.2019.8924302>
34. Moroz, L., Samoty, V., Horyachyy, O.: Modified fast inverse square root and square root approximation algorithms: The method of switching magic constants. *Computation* **9**(2), 21 (2021). <https://doi.org/10.3390/computation9020021>
35. Moroz, L., Samoty, V., Węgrzyn, M., Dzelendzyak, U.: Efficient floating-point square root and reciprocal square root algorithms. In: *2021 11th IEEE International conference on Intelligent Data Acquisition and Advanced Computing Systems: technology and applications (IDAACS)*, Cracow, Poland, 22–25 September, 2022, Vol. 1, pp. 552–559 (2022). <https://doi.org/10.1109/IDAACS53288.2021.9660872>
36. Mostefa, M.B., Boussaid, A., Khezzar, A.: FPGA-based algorithm for harmonic current mitigation. In: *2022 2nd International Conference on Advanced Electrical Engineering (ICAEE)*, Constantine, Algeria, 29–31 October, 2022, pp. 1–5 (2022). <https://doi.org/10.1109/ICAEE53772.2022.9962021>
37. Moursund, D.G.: Optimal starting values for Newton-Raphson calculation of $\sqrt{x}$. *Comm. ACM* **10**(7), 430–432 (1967). <https://doi.org/10.1145/363427.363454>
38. Muller, J.-M., Brisebarre, N., de Dinechin, F., Jeannerod, C.-P., Lefèvre, V., Melquiond, G., Revol, N., Stehlé, D., Torres, S.: *Handbook of Floating-Point Arithmetic*, 2nd edn. Basel, Switzerland, Birkhäuser (2018). <https://doi.org/10.1007/978-0-8176-4705-6>
39. Muller, J.-M.: Elementary functions and approximate computing. *Proc. IEEE* **108**(12), 2136–2149 (2020). <https://doi.org/10.1109/jproc.2020.2991885>
40. Piparo, D., Innocente, V., Hauth, T.: Speeding up HEP experiment software with a library of fast and auto-vectorisable mathematical functions. *J. Physics: Conf. Ser.* **513**(5), 052027 (2014). <https://doi.org/10.1088/1742-6596/513/5/052027>
41. Walczyk, C., Moroz, L., Cieřliński, J.: Improving the accuracy of the fast inverse square root by modifying Newton-Raphson corrections. *Entropy* **3**(1), 86 (2021). <https://doi.org/10.3390/e23010086>
42. Wang, S., Deng, X., Liu, W., Li, Y., Chen, S. Chen, Liu, L.: FPGA-based acceleration of structured light depth estimation. In: *2022 China Automation Congress (CAC)*, Xiamen, China, 25–27 November, 2022, pp. 4191–4196 (2022). <https://doi.org/10.1109/CAC57257.2022.10055770>
43. Wei, J., Kuwana, A., Kobayashi, H., Kubo, K., Tanaka, Y.: Floating-point inverse square root algorithm based on Taylor-series expansion. *IEEE Trans. Circ. Syst. II: Express Briefs* **68**(7), 2640–2644 (2021). <https://doi.org/10.1109/TCSII.2021.3062358>
44. Yasin, A., Pillement, T., Ciesielski, S.: Functional verification of hardware dividers using algebraic model. In: *2019 IFIP/IEEE 27th International conference on Very Large Scale Integration (VLSI-SoC)*, Cuzco, Peru, 6–9 October, 2019, pp. 257–262 (2019). <https://doi.org/10.1109/VLSISoC.2019.8920335>